

Recreating Scanned PDFs as High-Fidelity, Text-Selectable HTML: Production OCR Engines and Workflows for Arabic Complex Layouts

Executive summary

Recreating scanned PDFs as **HTML documents with selectable text and high layout fidelity** is best treated as a **document “sandwich” problem**: preserve the original page appearance (usually as a raster image) while overlaying a geometrically aligned text layer for selection, search, and copy/paste—analogue to “searchable PDF” but rendered in a browser. When Arabic right-to-left layout, multi-column pages, tables, marginalia, rotated/flipped pages, and mixed handwriting/print are present, “OCR alone” is not enough; **layout analysis + reading-order modelling + rigorous QA** is what determines whether the output is usable. ¹

A production-grade approach that consistently achieves high fidelity has three defining characteristics:

First, it uses a **canonical intermediate representation** that captures (a) per-page geometry (page size, coordinates), (b) a hierarchical structure (blocks → lines → words/tokens), (c) table structure when present, and (d) an explicit reading order graph/sequence. Mature standards such as **PAGE XML** (explicit reading order constructs) and **ALTO XML** (layout + OCR text) exist specifically for this role, and you can mirror them in a custom JSON schema. ²

Second, it chooses an OCR engine/workflow in which **Arabic and complex layouts are first-class**, not an afterthought. In practice this usually means (a) a commercial OCR SDK/server with strong Arabic support (e.g., ABBYY), or (b) a cloud document intelligence service with explicit reading-order output and Arabic handwriting support (notably Azure Document Intelligence), plus robust fallbacks for “worst pages”. ³

Third, it treats evaluation as an engineering discipline: measure **CER/WER**, reading order accuracy, layout overlap (IoU), and table structure similarity (e.g., TEDS), and continuously monitor drift. Without these metrics, “pretty HTML” often hides text-order failures that break downstream use. ⁴

Problem framing and design goals

High-fidelity “HTML recreation” of a scanned PDF can target three distinct fidelity modes, and selecting the right mode early prevents endless rework:

A **visual replica mode** renders each page as a fixed canvas (usually an `<img>` background) and overlays invisible or lightly visible positioned text spans. This maximises layout fidelity and is closest to searchable PDF behaviour, while staying robust to complex layouts. ⁵

A **hybrid mode** still uses page-fixed positioning, but groups spans into semantic containers (columns, headings, table cells) so you can support better selection behaviour, improved accessibility, and

structured exports. This depends on reliable layout analysis outputs (tables, regions, reading order).

6

A **reflowed/semantic HTML mode** attempts to recreate the document as responsive HTML (proper `<table>`, headings, paragraphs) without fixed coordinates. For “strict layout fidelity to scan”, this is typically a secondary derivative output rather than the primary deliverable, especially for Arabic multi-column content and low-quality scans.

Given your constraints—Arabic, complex layout, poor scans, handwriting, flipped pages, and strict reading order—the most reliable production strategy is usually:

Primary output: visual replica HTML (image + text overlay), guaranteed high fidelity.

Secondary output: semantic JSON/HTML extracted from the canonical representation where confidence is high (tables, headings, form fields). 7

Two specific failure modes deserve upfront attention:

Reading order is not inherently “solved” by OCR; it is a separate problem in layout analysis and can materially affect usability and evaluation. PAGE XML exists partly because reading order needs explicit representation beyond raw coordinates. 8

Arabic adds additional sensitivity: right-to-left directionality, BiDi behaviour with embedded Latin numerals, and frequent diacritics/ligatures make both OCR and rendering more brittle if you do not explicitly model direction and normalisation. Tools like Kraken explicitly address BiDi/RTL but also warn that “principal text direction” often cannot be inferred purely automatically because layout and document conventions matter. 9

Technology landscape and comparison

Comparison of OCR engines

The table below focuses on “production grade” capabilities relevant to your constraints: Arabic support, handwriting, tables, reading order, output formats, licensing, and operational integration.

Engine / service	Vendor	Arabic (print)	Arabic handwriting	Tables	Reading order support	Output formats & geometry	Licens
FineReader Server / FineReader Engine	ABBYY ¹⁰	Strong (explicit Arabic support; Arabic-focused options exist) ¹¹	“Hand-printed” ICR is supported; free-form Arabic handwriting varies by use case (often requires training/validation) ¹²	Table detection + layout reconstruction is a core feature set ¹³	Provides document structure outputs; reordering can be addressed in engine workflows/parameters but still needs validation ¹⁴	Server outputs include HTML, JSON, ALTO XML (plus PDF/PDF-A/Office formats) ¹⁵	Comm
Document AI (Enterprise Document OCR)	Google ¹⁶	Document OCR supports 200+ languages (Arabic included in practice, but validate on your corpus) ¹⁷	Handwritten text extraction is supported; vendor messaging highlights handwriting recognition (language coverage should be tested) ¹⁷	Rich document structure; tables/structure depend on processors and post-processing ¹⁸	Provides hierarchical elements (blocks/paragraphs/lines/tokens) with layout; strict reading order may require your own ordering logic ¹⁸	Canonical “Document” JSON with per-element layout info (bounding polygons) ¹⁹	Comm cloud
Azure AI Document Intelligence (Read + Layout)	Microsoft ²⁰	Arabic is supported for printed text ²¹	Arabic handwriting is explicitly listed as supported for Read and Layout handwritten extraction ²¹	Layout model outputs table structure with row/col spans + bounding polygons and spans into the “content” string ²²	Top-level content is explicitly “in reading order”; note: reading order is not provided across page boundaries ²³	JSON with polygons, spans, tables; optional searchable PDF output via add-on capability ²⁴	Comm cloud

Engine / service	Vendor	Arabic (print)	Arabic handwriting	Tables	Reading order support	Output formats & geometry	Licensing
PDF Services API (OCR PDF + Extract API)	Adobe ²⁵	OCR enables searchable PDFs; Extract outputs structured JSON for scanned or native PDFs (language coverage must be validated for Arabic) ²⁶	Depends on OCR subsystem; validate on Arabic handwriting specifically	Extract API explicitly targets text + complex tables + figures into structured JSON ²⁷	Reading order behaviour is not guaranteed for all complex layouts; requires evaluation on your documents	OCR creates searchable PDFs; Extract returns JSON structure ²⁶	Commercial cloud
Textract	Amazon Web Services ²⁸	Not suitable: language support in official FAQ is limited (does not include Arabic) ²⁹	Handwriting support is described as English-only in FAQ guidance ³⁰	Strong for tables/forms in supported languages ³¹	Reading order is output as blocks/relationships but Arabic constraint blocks adoption	JSON blocks/geometry; API ³²	Commercial cloud
Tesseract 5	—	Supports Arabic via trained data; quality depends heavily on scan quality and training/domain adaptation ³³	Not a dedicated Arabic handwriting engine; can be trained/fine-tuned but requires substantial GT ³⁴	Limited native table semantics (mostly text extraction + layout heuristics)	Provides OSD/PSM options; reading order in complex layouts often requires external layout analysis ³⁵	Text + hOCR and other outputs via tooling; geometry at word level in hOCR-style outputs ³⁶	Apache ³⁷
PaddleOCR (PP-OCR + PP-Structure)	—	Has an Arabic recognition model variant; multilingual OCR is a core focus ³⁸	Handwriting varies by model; typically stronger for printed + scene text than cursive manuscript handwriting	PP-Structure integrates layout analysis + table recognition in a document parsing pipeline ³⁹	Reading order can be addressed via structure parsing + your ordering rules/models	Structured document parsing components; custom pipelines ⁴⁰	Apache ⁴¹

Engine / service	Vendor	Arabic (print)	Arabic handwriting	Tables	Reading order support	Output formats & geometry	Licens
Kraken	—	Explicit RTL/BiDi support; widely used for non-Latin and historical scripts ⁴²	Strong path for handwriting via HTR-style training; designed for connected scripts ⁴³	Can be paired with PAGE XML / ALTO outputs; table semantics typically external	Includes layout analysis/reading order capabilities; still needs corpus-specific validation ⁴⁴	Outputs include ALTO and hOCR; supports PAGE XML in tooling ecosystem ⁴⁵	Apach ⁴⁶
Mistral OCR API	Mistral AI ⁴⁷	Designed to “understand” document elements (text/tables/media); language coverage must be validated for Arabic quality ⁴⁸	Can ingest images/PDFs and extract ordered content; handwriting quality must be tested per domain ⁴⁸	Explicitly claims table extraction and structured understanding ⁴⁸	Returns “ordered interleaved text and images”; strict reading order still needs evaluation/guardrails ⁴⁸	API returns text/markdown-like representation; geometry anchoring may be weaker than classic OCR unless you add it ⁴⁸	Comn cloud

Layout analysis, reading order, and standards

For strict reading order and geometry preservation, the “glue” around OCR matters as much as OCR itself.

PAGE XML is designed to represent page structure, including explicit reading order constructs, and is used widely in workflow ecosystems and ground-truth guidelines. ⁴⁹

ALTO XML, maintained as a standard (e.g., by the Library of Congress), is specifically for “layout and text objects” with coordinates and is common in digitisation pipelines. ⁵⁰

hOCR is an open standard encoding OCR + layout in an HTML subset, useful as an intermediate when your end target is HTML overlay rendering. ⁵¹

Open-source toolkits like **LayoutParser** provide a unifying interface for deep-learning layout detection, intended to streamline customisation for real-world document image analysis pipelines. ⁵²

Academic work and competitions emphasise that reading order is a first-class evaluation target. PRIMA’s work on reading order evaluation formalises why reading order errors are hard to measure and why region segmentation mismatches complicate evaluation. ⁵³

Comparison of conversion pipelines to HTML

Pipeline	What it optimises	Why it's relevant to you	Primary risks/limits	Licensing notes / ops
"OCR → fixed-layout HTML overlay" (custom renderer)	Maximum visual fidelity + selectable text	Best fit for strict layout fidelity and complex Arabic layouts; you fully control reading order and selection behaviour using your canonical JSON/PAGE-like structure ⁵⁴	Engineering complexity: you must build robust ordering, coordinate transforms, and QA UI; text-to-image alignment must be carefully managed	Your code; driven by chosen OCR engine
"OCR to searchable PDF → PDF-to-HTML" using OCRmyPDF → pdf2htmlEX	Fast path to selectable HTML (via PDF text layer)	OCRmyPDF adds a text layer to scanned PDFs; pdf2htmlEX converts PDFs to HTML while preserving text/format, often yielding high-fidelity positioned HTML ⁵⁵	pdf2htmlEX is powerful but produces complex HTML/CSS; quality depends on correctness of OCR text layer alignment; pdf2htmlEX is GPL	OCRmyPDF is open source; pdf2htmlEX GPLv3 ⁵⁶
"Azure Read searchable PDF add-on → PDF-to-HTML"	Managed OCR + standard PDF output	Azure can output "searchable PDF" with embedded text for scanned PDFs (Read model); you can then convert to HTML similarly ⁵⁷	Still need PDF-to-HTML conversion quality control; searchable PDF capability is model-scoped and API-specific ⁵⁸	Commercial cloud; good SDK support
"ABBYY export HTML/ALTO/JSON → HTML renderer"	Mature OCR + structure export	FineReader Server outputs HTML, JSON, ALTO; these intermediate outputs can drive a faithful HTML overlay renderer ¹⁵	Licensing + deployment overhead; you still need to test reading order for Arabic multi-column/table-heavy pages	Commercial software
"LLM/VLM OCR → Markdown/JSON → HTML" (your current style)	Rapid structure inference, flexible schema	VLM OCR systems can return ordered textual representations and capture tables; useful as a fallback or as a semantic enrichment layer ⁴⁸	Hallucination/normalisation risks; weaker geometry anchoring unless you explicitly preserve coordinates; requires strict QA gates	Commercial APIs; cost and stability vary

Recommended production architectures and schemas

Architecture pattern: canonical representation + fixed-layout HTML overlay

This is the most robust architecture when your primary deliverable is an HTML document that visually matches the scan and preserves strict reading order.

Key design choice: **store a page-accurate canonical representation** (PAGE-like JSON) as your system-of-record; generate HTML as a deterministic view.

```
flowchart TB
    A[PDF Ingest] --> B[Page Rasterisation<br/>300-400+ DPI as needed]
    B --> C[Preprocessing<br/>deskew • denoise • dewarp • contrast • binarise]
    C --> D[Orientation & Flip Detection<br/>0/90/180/270]
    D --> E[Layout Analysis<br/>regions • columns • tables • figures]
    E --> F[OCR / HTR<br/>word + line geometry + confidences]
    F --> G[Reading Order Resolution<br/>graph + sequence]
    G --> H[Post-processing<br/>Arabic normalisation • LM correction • table structuring]
    H --> I[(Canonical Store)<br/>JSON/PAGE-like + assets]
    I --> J[Fixed-layout HTML Renderer<br/>image background + text overlay]
    I --> K[Secondary Outputs<br/>semantic HTML • searchable PDF • JSON for downstream]
    J --> L[QA UI + Sampling Review]
    L --> I
```

Why this pattern aligns with your constraints:

You can explicitly model and enforce reading order (including region groups and nested ordering) in a way compatible with PAGE XML concepts. ⁵⁹

It gracefully handles “hard layout” documents: multi-column Arabic, tables, marginalia, diagrams—because your visual fidelity comes from the preserved raster page, not from fragile CSS reflow. ⁶⁰

It supports an iterative QA loop: manual correction updates the canonical representation; HTML regenerates deterministically. (OCR-D’s GT and QA philosophy reflects this workflow orientation.) ⁶¹

Strong candidate OCR/layout component sets for this pattern:

Commercial-first stack: ABBYY OCR + export (HTML/JSON/ALTO) + custom rendering. ⁶²

Cloud-first stack with strong reading order and Arabic handwriting: Azure Document Intelligence `prebuilt-layout` for structure + `prebuilt-read` for high-resolution OCR and optional searchable PDF output; use `content` spans as an initial reading-order signal and override when your evaluation indicates failures. ⁶³

Open-source controllable stack: PaddleOCR (PP-Structure) or Kraken + LayoutParser fine-tuned on in-domain pages; embed outputs into PAGE-like JSON, then render. ⁶⁴

Architecture pattern: searchable PDF as the “layout container”, then convert to HTML

If you want to reduce custom rendering effort, a pragmatic production path is: **produce a searchable PDF first**, then convert it to HTML.

```
flowchart LR
    A[Scanned PDF] --> B[OCR to Searchable PDF<br/>ABBYY or Azure Read or OCRmyPDF]
    B --> C[PDF-to-HTML Conversion<br/>pdf2htmlEX]
    C --> D[HTML Post-processing<br/>RTL fixes • selection tuning • metadata]
    D --> E[QA Sampling + Metrics]
```

What makes this attractive:

OCRmyPDF explicitly adds an OCR text layer to scanned PDFs (making them searchable/selectable) and can emit sidecar text; it is purpose-built for the “text layer over image” pattern. ⁶⁵

Azure can output a searchable PDF from `prebuilt-read` as an official add-on capability (downloadable as `application/pdf`). ⁵⁸

pdf2htmlEX is explicitly designed to convert PDFs to HTML “without losing text and format,” and is a common tool for faithful PDF web rendering. ⁶⁶

Critical caveat: pdf2htmlEX is GPLv3, which can be a blocker for some proprietary deployments unless you isolate it as a separate service and ensure licensing compliance. ⁶⁷

Canonical JSON schema example

A production schema that supports strict fidelity should be explicit about: coordinate systems, element hierarchy, reading order, confidence, and language/direction metadata.

```
{
  "document_id": "uuid",
  "source": {
    "filename": "scan.pdf",
    "page_count": 128
  },
  "pages": [
    {
      "page_index": 0,
      "image": {
        "uri": "pages/0000.webp",
        "width_px": 2480,
        "height_px": 3508,
        "dpi": 300
      },
      "rotation_deg": 0,
```



```

"blocks": [
  {
    "block_id": "b1",
    "type": "text|table|figure|header|footer",
    "bbox": [x0, y0, x1, y1],
    "direction": "rtl",
    "language": "ar",
    "lines": [
      {
        "line_id": "l1",
        "baseline": [[x,y], [x,y]],
        "bbox": [x0, y0, x1, y1],
        "tokens": [
          {
            "token_id": "t1",
            "text": "...",
            "bbox": [x0, y0, x1, y1],
            "confidence": 0.93
          }
        ]
      }
    ]
  }
],
"tables": [
  {
    "table_id": "tb1",
    "bbox": [x0, y0, x1, y1],
    "cells": [
      {
        "row": 0,
        "col": 0,
        "row_span": 1,
        "col_span": 2,
        "bbox": [x0, y0, x1, y1],
        "text_spans": [{"block_id": "b7", "line_id": "l22"}]
      }
    ]
  }
],
"reading_order": {
  "units": [
    {"unit_id": "b_header", "ref": {"block_id": "b3"}},
    {"unit_id": "b_body_col1", "ref": {"block_id": "b10"}},
    {"unit_id": "b_body_col2", "ref": {"block_id": "b11"}},
    {"unit_id": "tb1", "ref": {"table_id": "tb1"}}
  ],
  "sequence": ["b_header", "b_body_col1", "b_body_col2", "tb1"],
  "confidence": 0.82,
  "method": "engine|heuristic|model"
}

```

```

    }
  ]
}

```

This schema intentionally includes a reading order explicit sequence + provenance (“engine/heuristic/model”), reflecting why reading order evaluation is a dedicated subtask in document recognition pipelines. ⁶⁸

Fixed-layout HTML overlay example

A minimal but production-friendly approach uses:

an `<img>` (or CSS background) for the raster page, and
an overlay `<div>` containing absolutely positioned spans.

```

<div class="page" style="position:relative; width:2480px; height:3508px;">
  
  <div class="text-layer" style="position:absolute; inset:0;">
    <span
      class="tok"
      dir="rtl"
      style="position:absolute; left:1800px; top:420px; width:520px; height:
42px;
          font-size:38px; line-height:42px; opacity:0;">
      النص العربي هنا
    </span>
  </div>
</div>

```

Using a hidden text layer to preserve the original appearance is the same strategy that makes “searchable PDF” viable as a digitisation format. ⁶⁹

Evaluation methodology, metrics, and datasets

Metrics that measure “fidelity” rather than just text accuracy

A rigorous evaluation plan should treat fidelity as multi-dimensional:

Text accuracy:

Use **Character Error Rate (CER)** and **Word Error Rate (WER)** as defined in OCR-D’s QA specification. CER is edit-distance-based at the character level; WER is analogous at the word level. ⁷⁰

Layout accuracy:

For region detection (text blocks, tables, figures), use object-detection style overlap metrics (IoU-based matching, AP/mAP). DocLayNet provides baseline layout segmentation evaluation using standard detection metrics and emphasises layout diversity effects. ⁷¹

Reading order accuracy:

Use rank/sequence metrics such as **Kendall's Tau distance** over ordered block/line sequences. Benchmarks explicitly use Kendall Tau similarity for reading order detection evaluation. ⁷²

Table structure fidelity:

For table reconstruction, use **TEDS** (Tree-Edit-Distance-based Similarity), designed to penalise structural misalignment and OCR errors in HTML-table reconstructions. ⁷³

End-to-end visual fidelity (optional but valuable):

Render your HTML output to an image and compute a perceptual similarity metric such as **SSIM** to detect "layout drift" regressions, especially when CSS/renderer changes occur. ⁷⁴

Evaluation protocol

A production-friendly protocol that avoids misleading results:

Treat the canonical representation as ground truth (or PAGE XML GT), including explicit reading order. PAGE XML is designed to encode not only layout and text, but also reading order groupings, enabling reading-order-aware evaluation. ⁷⁵

Evaluate at multiple granularities: page, block, line, and token. Azure's API design (top-level `content` with spans to elements) illustrates a practical strategy: a single canonical "content string" in reading order with per-element span alignment. ⁷⁶

Separate "OCR accuracy" from "layout/reading order accuracy" where possible. Research on reading order evaluation highlights that segmentation mismatches can confound reading order scoring unless handled carefully. ⁷⁷

Test dataset suggestions for Arabic complex layouts

No single public dataset fully matches "Arabic + real-world scans + complex layout + handwriting". A realistic approach is to combine:

Arabic handwriting (page/line level):

Muharaf (historic handwritten Arabic pages with line polygons and PAGE XML ground truth) is directly aligned with your needs for handwriting + page geometry. ⁷⁸

KHATT provides large-scale handwritten Arabic forms (strong for handwriting recognisers, though not necessarily complex publication layout). ⁷⁹

IFN/ENIT provides Arabic handwritten word data; useful for sub-components, less for full page layout. ⁸⁰

MADCAT/OpenHaRT datasets (via LDC and related evaluation literature) are historically used for Arabic handwriting recognition evaluation; even if access is restricted, they define established task setups. ⁸¹

Arabic printed pages with layout complexity:

BCE-Arabic-v1 is explicitly described as scanned Arabic book pages with varied content/layout; it has been used in work on Arabic newspaper/book layout analysis. ⁸²

PRImA's RASM competition focuses on Arabic scientific manuscripts (marginalia, diagrams), which maps closely to “complex layout in Arabic manuscripts”. ⁸³

General layout and reading-order benchmarks to stress your layout subsystem:

DocLayNet (diverse layouts, COCO annotations) is valuable to pretrain/fine-tune layout detectors before specialising to Arabic. ⁷¹

ReadingBank provides large-scale reading order ground truth derived from Word documents and is useful for training reading order predictors. ⁸⁴

Tables:

PubTables-1M and PubTabNet (with released TEDS evaluation code) are valuable for table detection/structure pretraining, even if they are not Arabic-specific. ⁸⁵

Synthetic data generation strategies

Public Arabic datasets often underrepresent the “messy scan” reality. Two pragmatic strategies:

Template-driven generation: render Arabic documents (multi-column, tables, headers/footers) to PDF/PNG, then degrade (blur, noise, downsampling, skew, warping, background stains), and generate perfect ground truth directly from the renderer. Synthetic Arabic OCR datasets explicitly discuss the trade-offs of synthetic generation and what they may omit (e.g., complex multi-column structures, embedded figures). ⁸⁶

Hybrid synthesis from real layouts: take real scanned page images, mask text regions, re-render new Arabic text into those regions with matching font/size, then degrade again. This preserves “layout realism” (margins, ornaments, stamps) while expanding language coverage.

If you already have production PDFs, the most valuable “synthetic” asset is often **in-domain augmentation**: generate a spectrum of distortions that match your scan artefacts, then fine-tune OCR/layout models on that distribution.

Best practices for preprocessing and hard cases

Scan quality and resolution

Resolution matters nonlinearly. ABBYY's guidance recommends scanning at **300 dpi** for OCR, warns that extremely low resolution (e.g., <150 dpi) harms quality, and notes diminishing returns above ~600 dpi for typical documents. ⁸⁷

Tesseract documentation similarly notes it works best at **at least 300 dpi** and suggests rescaling when needed. ⁸⁸

For small Arabic fonts, ABBYY advises increasing scanning resolution (e.g., 400–600 dpi) when characters are too small for reliable recognition at 300 dpi. ⁸⁹

Deskew, denoise, dewarp, binarise

A robust workflow separates preprocessing into explicit stages (crop → binarise → deskew → dewarp → segment → recognise). OCR-D's workflow descriptions and QA philosophy provide a concrete stage model, including a dedicated dewarping stage that straightens curved lines and may require binarised input for some modules. ⁹⁰

Dewarping is not optional for many book scans (curved baselines). If you do not dewarp, Arabic baseline curvature can produce systematic substitution/segmentation errors and degrade reading order heuristics.

Handling flipped pages and rotation

Do not trust PDF metadata alone for orientation. Instead, combine:

OCR engine / toolkit orientation detection (e.g., Tesseract page segmentation modes that include OSD). ⁹¹

Text-line angle classification (PaddleOCR explicitly includes angle classification, supporting at least 0/180 in its built-in classifier documentation). ⁹²

A production strategy should record `rotation_deg` and `mirror` flags per page, and normalise coordinates into a consistent orientation before layout analysis and OCR.

Arabic directionality and BiDi safety

Always store Arabic text as Unicode and ensure:

HTML spans set `dir="rtl"` and (when mixing numerals/Latin) apply BiDi-safe rendering constructs (`unicode-bidi`, `<bdi>`, etc.) in the overlay layer.

Your canonical schema should record both **principal direction** (page/region) and **inline direction** (token-level), because inline BiDi rules can behave differently from region-level reading direction. Kraken's documentation explicitly distinguishes "principal text direction" from inline direction handled by the BiDi algorithm, and warns that principal direction often cannot be inferred purely automatically. ⁹³

Tables: detection, structure, and HTML reconstruction

If tables are a dominant layout component, prefer engines that explicitly output table structure:

Azure's layout model outputs tables including cell bounding polygons, row/column indices, spans, and merged cells; this is directly useful for reconstructing HTML tables or for building a table overlay layer. ²²

PaddleOCR's PP-Structure pipeline is explicitly designed to integrate layout analysis and table recognition in an end-to-end document parsing framework. ³⁹

For evaluation and regression testing, table structure similarity metrics like TEDS were proposed specifically to capture structural and OCR errors in table-to-HTML reconstruction. ⁷³

Handwriting: printed OCR vs HTR-style workflows

Arabic cursive handwriting often behaves more like **HTR (Handwritten Text Recognition)** than classic OCR. Three practical options:

Cloud route: Azure Document Intelligence explicitly lists Arabic as supported for handwritten text extraction (Read and Layout handwritten lists include Arabic). ²¹

Open HTR route: Kraken is designed for connected scripts and provides straightforward training workflows; it is specifically framed as trainable for many scripts without glyph-level segmentation. ⁹⁴

Dataset-driven route: use Muharaf (PAGE XML line polygons + expert transcriptions) for handwriting model training/evaluation, and use PAGE XML as your canonical ground truth format. ⁹⁵

Using LLMs/VLMs safely in a production OCR pipeline

Given your experiments (VLM OCR → Markdown/JSON → HTML), a robust production stance is:

Use LLM/VLM outputs as **semantic enrichment or fallback**, not as the canonical geometric truth, unless the model output is explicitly grounded to geometry and validated against it.

Mistral OCR positions itself as extracting ordered document content including text, tables, and images; this is valuable for semantic inference and “messy document understanding,” but should be guarded by geometry-based QA if layout fidelity is a hard requirement. ⁴⁸

A practical “LLM assist” pattern is: take deterministic OCR + layout outputs (tokens/boxes/tables), then ask an LLM to propose reading order or structured grouping—but force it to reference element IDs only, not generate new text.

Example prompt template (reading order repair):

```
You are given OCR blocks with IDs, bounding boxes, direction, and text.
Task: output a JSON object with a single field "sequence" listing the block
IDs
in the correct human reading order for this page.
Rules:
- Do not change any text.
- Do not invent new IDs.
- Respect Arabic RTL reading conventions.
- Prefer column-wise reading consistent with the layout.
Provide only valid JSON.
```

Pair this with automated checks: sequence must include all blocks exactly once; adjacent swaps should be explainable by spatial relations; and Kendall Tau distance against a labelled test set should improve, not regress. ⁹⁶

Open-source vs commercial trade-offs and prioritised recommendations

A useful way to choose is to decide what you need to “own” versus what you can “buy”.

If **Arabic handwriting and explicit reading-order text output** are central, Azure Document Intelligence stands out because it (a) explicitly represents top-level `content` in reading order via spans, and (b) explicitly lists Arabic among supported handwritten extraction languages for Read/Layout. ⁹⁷

If **Arabic OCR accuracy on poor-quality scans** is the primary risk, ABBYY's FineReader Engine explicitly supports Arabic/Farsi and provides “end-to-end recognition for Arabic” intended to improve results on low-quality images; FineReader Server also supports rich export formats (HTML/JSON/ALTO), which aligns well with HTML recreation workflows. ⁹⁸

If **deployment control and custom training** are key, Kraken and PaddleOCR provide strong open-source foundations (Apache 2.0 licensing for both), but you should budget engineering time for training, layout modelling, and QA tooling. ⁹⁹

If you want to **leverage searchable PDF** as an intermediate to reach HTML faster, OCRmyPDF + pdf2htmlEX can be very effective, but pdf2htmlEX's GPLv3 licensing is often a decisive constraint in proprietary systems. ¹⁰⁰

Finally, regardless of engine choice, treat “best way” as an **evaluation-driven workflow**: build a representative Arabic complex-layout test suite (mixing datasets like Muharab/BCE-Arabic-v1/RASM-style pages plus your own scans), measure CER/WER, reading order, and table structure fidelity, and only then lock the production architecture. ¹⁰¹

¹ ⁹⁰ OCR-D Workflow Guide

https://ocr-d.de/en/workflows?utm_source=chatgpt.com

² ⁴⁷ ⁴⁹ ⁷⁵ The PAGE (Page Analysis and Ground-truth Elements) ...

https://www.primaresearch.org/www/assets/papers/ICPR2010_Pletschacher_PAGE.pdf?utm_source=chatgpt.com

³ ¹¹ ²⁰ ⁹⁸ OCR for Arabic and Farsi - Help Center - ABBYY

https://support.abbyy.com/hc/en-us/articles/360016365800-OCR-for-Arabic-and-Farsi?utm_source=chatgpt.com

⁴ ⁷⁰ Quality Assurance in OCR-D

https://ocr-d.de/en/spec/ocrd_eval.html?utm_source=chatgpt.com

⁵ ³⁶ ⁵¹ ⁶⁰ hOCR - OCR Workflow and Output embedded in HTML

https://kba.github.io/hocr-spec/1.2/?utm_source=chatgpt.com

⁶ ¹⁶ ²² What is the Document Intelligence layout model?

https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/prebuilt/layout?view=doc-intel-4.0.0&utm_source=chatgpt.com

⁷ ¹⁰ ²³ ²⁴ ⁶³ ⁷⁶ ⁹⁷ Document Intelligence APIs analyze document response

https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/concept/analyze-document-response?view=doc-intel-4.0.0&utm_source=chatgpt.com

⁸ ⁵³ ⁶⁸ ⁷⁷ The Significance of Reading Order in Document ...

https://primaresearch.org/www/assets/papers/ICDAR2013_Clausner_ReadingOrder.pdf?utm_source=chatgpt.com

- 9 42 45 **kraken documentation**
https://kraken.re/2.0.0/index.html?utm_source=chatgpt.com
- 12 **OCR, ICR, and other recognition technologies**
https://www.abbyy.com/ocr-sdk/features/ocr/?utm_source=chatgpt.com
- 13 **ABBYY® FineReader Engine 12 - WIN, Linux, and OS X**
https://static3.abbyy.com/abbyycommedia/32583/brochure-finereaderengine-feature-overview-en.pdf?utm_source=chatgpt.com
- 14 **How to reorder the recognized text**
https://support.abbyy.com/hc/en-us/articles/5177071847443-How-to-reorder-the-recognized-text?utm_source=chatgpt.com
- 15 62 **FineReader Server 14 Specifications - Help Center - ABBYY**
https://support.abbyy.com/hc/en-us/articles/360007070419-FineReader-Server-14-Specifications?utm_source=chatgpt.com
- 17 **Processor list | Document AI**
https://docs.cloud.google.com/document-ai/docs/processors-list?utm_source=chatgpt.com
- 18 **Handle processing response | Document AI**
https://docs.cloud.google.com/document-ai/docs/handle-response?utm_source=chatgpt.com
- 19 **Document AI | Google Cloud Documentation**
https://docs.cloud.google.com/document-ai/docs/reference/rest/v1/Document?utm_source=chatgpt.com
- 21 **Language and locale support for Read and Layout document analysis - Document Intelligence - Foundry Tools | Microsoft Learn**
<https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/language-support/ocr?view=doc-intel-4.0.0>
- 25 35 88 91 **Improving the quality of the output - Tesseract documentation**
https://tesseract-ocr.github.io/tessdoc/ImproveQuality.html?utm_source=chatgpt.com
- 26 **PDF Services API — OCR PDF**
https://www.adobe.io/document-services/apis/pdf-services/ocr-pdf/?utm_source=chatgpt.com
- 27 **PDF Extract API | Adobe PDF Services**
https://developer.adobe.com/document-services/docs/overview/pdf-extract-api/?utm_source=chatgpt.com
- 28 71 **DocLayNet: A Large Human-Annotated Dataset for Document-Layout Analysis**
https://arxiv.org/abs/2206.01062?utm_source=chatgpt.com
- 29 30 **Amazon Textract FAQs - AWS**
https://aws.amazon.com/textract/faqs/?utm_source=chatgpt.com
- 31 32 **Best Practices - Amazon Textract**
https://docs.aws.amazon.com/textract/latest/dg/textract-best-practices.html?utm_source=chatgpt.com
- 33 **Traineddata Files for Version 4.00 + | tessdoc - Tesseract OCR**
https://tesseract-ocr.github.io/tessdoc/Data-Files.html?utm_source=chatgpt.com
- 34 **How to train LSTM/neural net Tesseract | tessdoc**
https://tesseract-ocr.github.io/tessdoc/tess5/TrainingTesseract-5.html?utm_source=chatgpt.com
- 37 **Tesseract Open Source OCR Engine (main repository)**
https://github.com/tesseract-ocr/tesseract?utm_source=chatgpt.com
- 38 **General Table Recognition V2 - PaddleX Documentation**
https://paddlepaddle.github.io/PaddleX/3.3/en/pipeline_usage/tutorials/ocr_pipelines/table_recognition_v2.html?utm_source=chatgpt.com
- 39 40 64 **PaddleOCR 3.0 Technical Report**
https://arxiv.org/html/2507.05595v1?utm_source=chatgpt.com

- 41 **PaddleOCR/LICENSE at main**
https://github.com/PaddlePaddle/PaddleOCR/blob/main/LICENSE?utm_source=chatgpt.com
- 43 94 **Training a kraken model**
https://kraken.re/2.0.0/training.html?utm_source=chatgpt.com
- 44 **kraken documentation**
https://kraken.re/4.3.0/index.html?utm_source=chatgpt.com
- 46 99 **kraken/LICENSE at main**
https://github.com/mittagessen/kraken/blob/main/LICENSE?utm_source=chatgpt.com
- 48 **Mistral OCR**
https://mistral.ai/news/mistral-ocr?utm_source=chatgpt.com
- 50 **ALTO Technical Metadata for Layout and Text Objects**
https://www.loc.gov/standards/alto/?utm_source=chatgpt.com
- 52 **LayoutParser: A Unified Toolkit for Deep Learning Based ...**
https://arxiv.org/abs/2103.15348?utm_source=chatgpt.com
- 54 59 **ReadingOrder**
https://ocr-d.de/en/gt-guidelines/trans/lyLeserichtung.html?utm_source=chatgpt.com
- 55 65 69 100 **ocrmypdf - add an OCR text layer to PDF files**
https://manpages.ubuntu.com/manpages/jammy/man1/ocrmypdf.1.html?utm_source=chatgpt.com
- 56 67 **pdf2htmlEX/LICENSE at master**
https://github.com/coolwanglu/pdf2htmlEX/blob/master/LICENSE?utm_source=chatgpt.com
- 57 58 **Add-on capabilities - Document Intelligence - Azure AI services | Azure Docs**
<https://docs.azure.cn/en-us/ai-services/document-intelligence/concept/add-on-capabilities?view=doc-intel-4.0.0>
- 61 **The Ground Truth Guidelines**
https://ocr-d.de/en/gt-guidelines/trans/?utm_source=chatgpt.com
- 66 **pdf2htmlEX - converts PDF to HTML without losing text and ...**
https://manpages.ubuntu.com/manpages/xenial/man1/pdf2htmlEX.1.html?utm_source=chatgpt.com
- 72 96 **\readoc: A Unified Benchmark for Realistic Document ...**
https://arxiv.org/html/2409.05137v1?utm_source=chatgpt.com
- 73 **Image-based table recognition: data, model, and evaluation**
https://arxiv.org/abs/1911.10683?utm_source=chatgpt.com
- 74 **Image Quality Assessment: From Error Visibility to Structural ...**
https://www.cns.nyu.edu/pub/lcv/wang03-preprint.pdf?utm_source=chatgpt.com
- 78 95 101 **Muharaf: Manuscripts of Handwritten Arabic Dataset for ...**
https://arxiv.org/html/2406.09630v1?utm_source=chatgpt.com
- 79 **KHATT: An open Arabic offline handwritten text database**
https://www.sciencedirect.com/science/article/abs/pii/S0031320313003300?utm_source=chatgpt.com
- 80 **IFN/ENIT - database Arabic OCR handwritten arabic word ...**
https://www.ifnenit.com/?utm_source=chatgpt.com
- 81 **Linguistic Resources for Handwriting Recognition and ...**
https://aclanthology.org/L12-1463/?utm_source=chatgpt.com
- 82 **BCE-Arabic-v1 dataset | Proceedings of the 9th ...**
https://dl.acm.org/doi/10.1145/2910674.2910725?utm_source=chatgpt.com

83 **PRImA | RASM2018**

https://www.primaresearch.org/RASM2018/?utm_source=chatgpt.com

84 **Pre-training of Text and Layout for Reading Order Detection**

https://aclanthology.org/2021.emnlp-main.389.pdf?utm_source=chatgpt.com

85 **PubTables-1M: Towards comprehensive table extraction from unstructured documents**

https://arxiv.org/abs/2110.00061?utm_source=chatgpt.com

86 **SARD: A Large-Scale Synthetic Arabic OCR Dataset for ...**

https://arxiv.org/html/2505.24600v1?utm_source=chatgpt.com

87 **Scanning tips**

https://help.abbyy.com/en-us/finereader/15/user_guide/scangeneral?utm_source=chatgpt.com

89 **Source image recommendations**

https://support.abbyy.com/hc/en-us/articles/360017326979-Source-image-recommendations?utm_source=chatgpt.com

92 **Text Direction Classification - PaddleOCR Documentation**

https://www.paddleocr.ai/latest/en/version2.x/ppocr/model_train/angle_class.html?utm_source=chatgpt.com

93 **Advanced Usage — kraken documentation**

https://kraken.re/5.3.0/advanced.html?utm_source=chatgpt.com